Cypress + TypeScript interview questions, answered with this framework

These are Cypress, TypeScript, API testing and framework-design questions that come up in SDET and test automation interviews. Each answer has a short explanation and, where useful, an example. Each one ends with an **In the framework** pointer: the file in this repository that does it, or the reason the framework does not use it. Links go to github.com/niranjankagri/cypress-typescript-framework.

Versions: Cypress 16.2, TypeScript 6.0, React 19, Vite 8, zod 4, Node 24, cypress-mochawesome-reporter 5.

Contents

1. Cypress fundamentals (Q1-Q12)
2. Commands, queries and retry-ability (Q13-Q22)
3. Selectors, actions and assertions (Q23-Q32)
4. Network control with cy.intercept (Q33-Q42)
5. Time, sessions and browser state (Q43-Q50)
6. API testing with cy.request (Q51-Q57)
7. Component testing (Q58-Q63)
8. Configuration, TypeScript and CI (Q64-Q72)
9. Flaky tests, debugging and reporting (Q73-Q78)
10. Framework design and scenarios (Q79-Q86)
11. Quick reference

---

1. Cypress fundamentals

Q1. How is Cypress built, and how does that differ from Selenium?

Cypress runs **inside the browser**: the test code and the application run in the same browser, in two frames, controlled by a Node process that also acts as a proxy for all network traffic. Selenium (and WebDriver BiDi tools) run the test in a separate process and send commands to the browser over a protocol.

	Selenium WebDriver	Cypress
Where tests run	separate process, any language	in the browser, JavaScript/TypeScript
Waiting	explicit/implicit waits written by you	built-in retrying of queries and assertions
Network	no built-in control	`cy.intercept`: spy, delay, stub any request
Clock	no	`cy.clock` / `cy.tick`
Browsers	all major, incl. Safari	Chrome family, Firefox, Electron (WebKit experimental)
Tabs, multiple browsers	yes	one tab; other origins via `cy.origin`
Debugging	logs, screenshots	time-travel command log, DOM snapshots

In the framework: everything; the network and clock tests (`cypress/e2e/ui/network.cy.ts`, `time.cy.ts`) are the kind of test that is hard to write with Selenium.

Q2. What runs in Node and what runs in the browser?

- **Browser:** spec files, support files, custom commands, page objects: everything you `import` into a spec is bundled and loaded in the browser.
- **Node:** `cypress.config.ts` and its `setupNodeEvents`, `cy.task()` handlers, file access (`cy.readFile`, `cy.writeFile`), the network proxy, `cy.request` (sent from Node, so no CORS), reporters and plugins.

That is why a secret should not be imported into a spec: it would end up in the browser bundle.

In the framework: `cypress.config.ts` (Node: reporter plugin, spec exclusion); `cypress/support/*` (browser).

Q3. `cypress open` vs `cypress run`?

`open` starts the interactive app: choose specs and a browser, watch them run, re-run on save, time-travel through commands. `run` is headless (by default Electron) for CI: prints a summary, takes screenshots of failures, writes reports and exits with the number of failed tests as the exit code.

In the framework: `npm run cy:open`; `npm test`, `cy:api`, `cy:ui`, `cy:component` use `run`.

Q4. What are the testing types in Cypress?

E2E tests visit a running app (`cy.visit`, `cy.request`). **Component** tests mount one component (`cy.mount`) using the project's dev server (Vite or webpack), without a backend. One config file holds both (`e2e` and `component` sections).

In the framework: both: `cypress/e2e/**` and `cypress/component/**`.

Q5. What is a spec, and how are tests organised?

A spec is one test file. Cypress uses Mocha's structure: `describe` / `context` group tests, `it` is a test, `before` / `beforeEach` / `afterEach` / `after` are hooks. Specs run one after another in one browser; each spec gets a fresh bundle (module state is reset between specs).

In the framework: one spec per feature (`auth`, `tasks`, `validation`, `login`, `filters`, `network`, `time`), `context` blocks for "as alice" / "as carol" in `cypress/e2e/ui/tasks.cy.ts`.

Q6. What does test isolation mean in Cypress?

With `testIsolation: true` (the default) Cypress clears cookies, localStorage and sessionStorage and visits a blank page **before every test**. A test cannot depend on what the previous one left in the browser. Server data is not touched: that is your job.

In the framework: browser isolation is the default; server data is reset with `cy.resetData()` in every `beforeEach`.

Q7. Why should tests not depend on each other?

Order-dependent tests fail in groups, cannot be run alone (`it.only`, a single spec, a retry) and hide the real cause. Each test should create what it needs and start from known data.

In the framework: every spec resets the data; setup data is created per test through the API (`cy.createTask`, `buildTask()` with unique titles).

Q8. Which browsers does Cypress support?

Chrome and Chromium-based browsers (Chrome, Edge, Electron, which ships with Cypress) and Firefox. WebKit support is experimental. Choose with `--browser chrome|firefox|edge|electron`.

In the framework: CI runs the UI specs in Chrome, Firefox and Edge (`.github/workflows/ci.yml`).

Q9. What are Cypress's main limitations?

- One browser tab; links with `target="_blank"` are tested by checking the `href` or removing the target.
- Another origin in the same test needs `cy.origin()`.
- No native mouse hover (use `.trigger('mouseover')` or a plugin that uses CDP).
- iframes need a helper (`its('0.contentDocument.body')`).
- JavaScript/TypeScript only.
- Parallel runs across machines are a paid Cypress Cloud feature (or a self-built split).

In the framework: none of these is needed by TaskBoard; parallelism is done with CI jobs per browser.

Q10. Cypress or Playwright: how do you choose?

Both are modern and reliable. Cypress: excellent interactive runner and time-travel debugging, component testing, simple mental model, big plugin ecosystem; but one tab, no WebKit (stable), parallelism via Cloud. Playwright: all three engines, tabs, multiple contexts, free parallel workers, more languages. Choose by the team's needs: a front-end team writing component and E2E tests in a React app often enjoys Cypress; cross-browser and multi-tab needs point to Playwright.

In the framework: my portfolio has both: this repo and `playwright-typescript-framework` test similar features, so the two styles can be compared.

Q11. What is the command log and time travel?

In `cypress open` every command and assertion is listed on the left. Hovering a command shows a DOM snapshot from that moment, with before/after for actions; clicking it prints details (element, request, yielded subject) to DevTools. This replaces most `console.log` debugging.

In the framework: `docs/images/failure-screenshot.png` shows the command log of a failed test.

Q12. How do you skip or focus tests?

`it.skip` / `describe.skip` skip; `it.only` runs only that test (never commit it: lint can forbid it). For conditional skipping use `this.skip()` inside a `function () {}` test, or decide with a config value.

In the framework: the report demo has an `it.skip` (`cypress/e2e/demo/report-demo.cy.ts`); the demo spec itself is excluded unless `--expose demo=true`.

---

2. Commands, queries and retry-ability

Q13. Are Cypress commands promises?

No. A command like `cy.get('button')` does not run when the line runs: it is **added to a queue**, and Cypress runs the queue after the test function returns. Commands yield a subject to the next command. That is why you cannot `await` them or assign their result:

```
const button = cy.get('button'); // wrong: this is a Chainable, not an element
cy.get('button').then(($button) => {
  /* here you have the element */
});
```

In the framework: ESLint's `cypress/no-assigning-return-values` rule is set to error (`eslint.config.js`).

Q14. What is retry-ability?

Queries (`get`, `find`, `contains`, `its`, `invoke` and others) and the assertions after them are **retried** until the assertion passes or the timeout (`defaultCommandTimeout`, 4 s by default) runs out. So `cy.getBySel('toast').should('have.text', 'Task added')` waits for the toast without any explicit wait. Only the last query chain before the assertion is retried.

In the framework: almost every UI assertion relies on it; the timeout is 6 s (`cypress.config.ts`).

Q15. Which commands are not retried?

Actions (`click`, `type`, `select`, `check`) and other non-query commands (`request`, `wait`, `task`) run once. Actions do wait for **actionability** first: the element must be attached, visible, enabled, not animating and not covered.

In the framework: `boardPage.addTask()` clicks without waits; Cypress waits until the button is actionable.

Q16. Queries vs commands since Cypress 12: why does it matter?

Queries are retried as a **chain**: if `cy.get('.list').find('li')` finds that `.list` was re-rendered (detached from the DOM), Cypress re-runs the whole chain instead of failing with "detached from the DOM". Actions are not part of that, so a chain that continues after an action (`.clear().type()`) can still hit a re-rendered element.

In the framework: `fill()` in `cypress/pages/BoardPage.ts` splits `clear()` and `type()` into two chains; ESLint's `cypress/unsafe-to-chain-command` enforces it.

Q17. `.then()` vs `.should(callback)`?

`.should(cb)` is retried: the callback runs again until it does not throw, so it must not change anything. `.then(cb)` runs once and can do anything (call other commands, compute values). Use `should` for assertions on things that settle, `then` to work with a value.

```
cy.getBySel('task-title').should(($titles) => expect($titles).to.have.length(4)); // retried
cy.apiToken('alice').then((token) => api.listTasks(token)); // once
```

In the framework: `.then` for tokens and created tasks (`cypress/e2e/api/tasks.cy.ts`); `should` for UI checks.

Q18. What are aliases?

`.as('name')` names a subject, a route or a stub. Use it with `cy.get('@name')` or `cy.wait('@name')`. For intercepts, `@name.all` gives every matched request. Aliases are reset before each test.

In the framework: `cy.intercept(...).as('create')` , then `cy.wait('@create')` or `cy.get('@create.all').should('have.length', 0)` ; `cy.stub().as('toggle')` in component tests.

Q19. How do you use a value from one command in a later one?

Inside `.then()` (closure), with an alias, or with a variable assigned **inside** a `.then()` and used in a later command block. Do not read a variable outside the chain before the command has run.

In the framework: `cy.createTask(...).then((created) => { ... created.id ... })` in `cypress/e2e/api/tasks.cy.ts` ; the password from `cy.env()` in `before()` in `auth.cy.ts` .

Q20. What is `cy.wrap()` for?

It turns a plain value or a promise into a Cypress subject, so you can chain commands and assertions on it, and Cypress waits for a promise to resolve.

In the framework: `cy.apiToken` returns the cached token with `cy.wrap(cached)` (`cypress/support/commands.ts`).

Q21. How do you write a custom command, and type it?

`Cypress.Commands.add('name', fn)` in a support file, plus a declaration in `declare global { namespace Cypress { interface Chainable { name(...): Chainable<T> } } }` . Use commands for cross-cutting steps (login, reset, data setup); keep screen logic in page objects.

In the framework: `getBySel` , `resetData` , `apiToken` , `loginAs` , `createTask` in `cypress/support/commands.ts` ; `mount` in `component.ts` .

Q22. What does `cy.task()` do?

Runs code in Node (registered in `setupNodeEvents`): reading a database, generating files, calling a service that the browser cannot reach. It yields what the task returns.

In the framework: not needed: the server has an HTTP reset hook, called with `cy.request` . With a real database, `cy.task('db:seed')` would be the usual way.

---

3. Selectors, actions and assertions

Q23. Which selectors do you use, and why `data-cy` ?

Attributes made for tests (`data-cy` , `data-test`) do not change when CSS classes, texts or layout change, and they say clearly "a test uses this". Text (`cy.contains`) is fine when the text is the requirement. Avoid long CSS paths and generated class names.

In the framework: `data-cy` on every element the tests use; `cy.getBySel()` wraps the lookup.

Q24. `cy.get` vs `cy.find` vs `cy.contains` vs `within`?

`cy.get` searches the whole document (or the `within` scope); `.find` searches inside the previous subject; `cy.contains(selector, text)` finds by text, optionally limited to a selector, and accepts a regular expression; `.within(() => ...)` scopes all commands inside it to one element.

In the framework: `boardPage.task(title)` uses `cy.contains` with an anchored regular expression and `.parents('[data-cy="task"]')`; `.within()` checks priority and due date of one row.

Q25. How do you check a list in order?

Read the texts and compare the array: `cy.get(sel).then(($els) => Cypress._.map($els, 'innerText'))` `.should('deep.equal', [...])`. Because `.then` is not retried, the list should be loaded first (or use a `should` callback).

In the framework: "lists the tasks newest first" in `cypress/e2e/ui/tasks.cy.ts`.

Q26. What assertion libraries does Cypress include?

Chai (`expect`, `should`), Chai-jQuery (`have.class`, `be.visible`, `have.attr`), Sinon-Chai for stubs and spies (`have.been.calledOnceWith`). `should('have.text', ...)` is the chainer form; `expect(...)` is used inside callbacks.

In the framework: all three: chainers in UI specs, `expect` in API specs, Sinon-Chai in component specs.

Q27. How do you handle `window.confirm` and `window.alert`?

Cypress accepts confirms automatically. To check the text or cancel, listen to `window:confirm`; returning `false` clicks Cancel. With several listeners, any `false` cancels, so for "cancel, then OK" use one stub:

```
const confirm = cy.stub().as('confirm');
confirm.onFirstCall().returns(false).onSecondCall().returns(true);
cy.on('window:confirm', confirm);
```

In the framework: delete tests in `cypress/e2e/ui/tasks.cy.ts` and `cypress/component/TaskItem.cy.tsx`.

Q28. How do you type special keys, dates and selects?

`type('{enter}')`, `{esc}`, `{selectall}`, `{backspace}`; `select('high')` for `<select>`; date inputs take `YYYY-MM-DD` with `type`. `type(text, { log: false })` keeps a password out of the command log; `{ delay: 0 }` types long texts fast.

In the framework: rename with `{enter}`, cancel with `{esc}`, dates in `addTask()`, password with `log: false` in `cypress/pages/LoginPage.ts`.

Q29. Why is `{ force: true }` a smell?

It skips the actionability checks: the test clicks things a user could not click (hidden, covered, disabled) and passes while the UI is broken. Fix the reason (scroll, wait for the overlay to go) instead.

In the framework: `cypress/no-force` is an ESLint error.

Q30. How do you check that something does NOT exist or did NOT happen?

`should('not.exist')` waits until the element is gone (or confirms it is absent). For "no request was sent", count the intercepts: `cy.get('@create.all').should('have.length', 0)`. For "callback not called", `cy.get('@stub').should('not.have.been.called')`. Put such checks after something that proves the app had time to act.

In the framework: client-side title check (`tasks.cy.ts`), Escape cancels editing, `TaskItem` rename with an unchanged title.

Q31. How do you test uploads and downloads?

Uploads: `cy.get('input[type=file]').selectFile('cypress/fixtures/file.pdf')` (also drag-and-drop with `{ action: 'drag-drop' }`). Downloads land in `cypress/downloads`; read them with `cy.readFile`.

In the framework: TaskBoard has no files; the folders are in `.gitignore` ready for it.

Q32. How do you test responsive layouts?

`cy.viewport(375, 667)` or a preset (`'iphone-8'`), per test or in `beforeEach`; the default comes from `viewportWidth` / `viewportHeight`.

In the framework: default 1280 x 800 in `cypress.config.ts`; a mobile pass is an easy extension.

---

4. Network control with cy.intercept

Q33. What can `cy.intercept` do?

All traffic of the app goes through Cypress's proxy, so a route can:

1. **spy**: let the request through, record it, wait for it and assert on request and response;
2. **change**: modify the request or the response, or delay it;
3. **stub**: answer instead of the server (status, body, headers, fixture, network error).

In the framework: all three in `cypress/e2e/ui/network.cy.ts`.

Q34. How do you wait for a request without `cy.wait(ms)`?

Alias the route and wait on it: `cy.intercept('GET', '/api/tasks*').as('list'); ...; cy.wait('@list')`. The wait yields the interception, so assert on it right away:

```
cy.wait('@create').its('request.body').should('deep.equal', { title, priority: 'high', dueDate });
```

In the framework: "adds a task with priority and due date" (`tasks.cy.ts`), query parameters (`filters.cy.ts`).

Q35. How do you stub a response?

```
cy.intercept('GET', '/api/tasks*', { fixture: 'tasks.json' });
cy.intercept('POST', '/api/tasks', { statusCode: 400, body: { error: '...', field: 'title' } });
cy.intercept('GET', '/api/tasks*', { forceNetworkError: true });
```

In the framework: fixture list, server validation message, network error, 401 in `cypress/e2e/ui/network.cy.ts`.

Q36. How do you stub only the first call?

Add `times: 1` to the route matcher: the first request is stubbed, later ones reach the server. Useful for "error, then Retry works".

In the framework: "shows an error banner on a server error and recovers with Retry".

Q37. How do you test a loading state?

Delay the real response so the state stays on screen long enough:

```
cy.intercept('GET', '/api/tasks*', (req) => {
  req.on('response', (res) => res.setDelay(1500));
}).as('list');
```

Then assert the spinner, `cy.wait('@list')`, and assert it is gone.

In the framework: loading state (`network.cy.ts`), busy login button (`login.cy.ts`).

Q38. When several intercepts match one request, which wins?

The most recently defined route is checked first. A handler can pass the request on with `req.continue()` or let it fall through to earlier routes; a static response ends the matching. So define general routes first (in `beforeEach`) and override them in a test.

In the framework: each test defines its own routes; the order rule matters once routes are shared.

Q39. Should UI tests stub the backend?

Both kinds are useful. Tests against the real API prove the system works together; stubbed tests cover states that are hard or slow to produce (errors, timeouts, empty or huge lists, expired login) and test the UI alone. A good suite has mostly real-backend flows plus targeted stubs. Stubs must follow the real contract, or they test a fiction.

In the framework: `tasks.cy.ts`, `filters.cy.ts`, `login.cy.ts` use the real API; `network.cy.ts` stubs; the fixture `cypress/fixtures/tasks.json` follows the same shape that the zod contract checks.

Q40. How do you check request headers, e.g. authentication?

`cy.wait('@me').its('request.headers.authorization').should('match', /^Bearer .../)`. Header names are lower-case in the interception.

In the framework: "sends the token as a Bearer header on every API call".

Q41. Intercept vs `cy.request`?

`cy.intercept` watches or changes requests the **app** makes. `cy.request` makes a request **from the test** (from Node, so no CORS and no app involved): for API tests and fast setup.

In the framework: `cypress/support/api.ts` (cy.request) vs `network.cy.ts` (intercept).

Q42. How do you test what happens when a token expires?

Either make the server refuse it (an unknown token in localStorage) or stub the API's 401. Assert that the app sends the user to the login page and removes the token.

In the framework: "goes back to the login page when the stored token is no longer valid" (`login.cy.ts`) and "sends the user to the login page when the API says the login expired" (`network.cy.ts`).

---

5. Time, sessions and browser state

Q43. How does `cy.clock` work?

It replaces the browser's `Date` , `setTimeout` , `setInterval` (and friends) with fake versions. Call it **before** `cy.visit` so the page starts with the fake clock. `cy.tick(ms)` moves time forward and runs due timers. You can replace only some functions:

```
cy.clock(new Date(2026, 9, 2, 12).getTime(), ['Date']); // fixed "today", real timers
cy.clock(Date.now(), ['setTimeout', 'clearTimeout']); // real date, controlled timers
```

In the framework: `cypress/e2e/ui/time.cy.ts` and the debounce test in `filters.cy.ts` .

Q44. How do you test a debounce?

Freeze the timers, type, `cy.tick(299)` and prove no request was sent, `cy.tick(1)` and wait for the request. Exact and instant, no real waiting.

In the framework: "searches only after typing stops for 300 ms" (`filters.cy.ts`).

Q45. How do you make date-dependent tests stable?

Never depend on the real date: freeze `Date` at a chosen day, use **noon** so no time zone shifts the date, and test the boundaries (the due date itself, the day after, a done task).

In the framework: three Overdue tests in `time.cy.ts` and two in `cypress/component/TaskItem.cy.tsx` .

Q46. What is `cy.session` ?

It caches a login: the first call runs the setup and saves cookies, localStorage and sessionStorage under an id; later calls restore them instead of logging in again. `validate()` checks that a restored session still works; if not, setup runs again. `cacheAcrossSpecs: true` shares it between specs.

In the framework: `cy.loginAs()` in `cypress/support/commands.ts` ; validate calls `/api/me` .

Q47. How should most tests log in?

Through the API (or by setting the token), not through the login form: faster and independent of the login screen. Test the form itself once, in its own tests.

In the framework: the form is tested in `login.cy.ts` ; all other UI tests use `cy.loginAs()` with an API login.

Q48. How do you set localStorage before the app starts?

`cy.visit(url, { onBeforeLoad(win) { win.localStorage.setItem(...) } })` , or log in with `cy.session` , which restores storage before the visit.

In the framework: the invalid-token test in `login.cy.ts` uses `onBeforeLoad` .

Q49. How do you test two users?

One test, two API tokens: act as one user, check as the other. In the UI, switch with `cy.loginAs('bob')` (each session has its own id).

In the framework: "answers 404 for another user's task" (alice tries, bob's task unchanged) in `cypress/e2e/api/tasks.cy.ts`.

Q50. How do you handle another origin (SSO login page)?

`cy.origin('https://login.example.com', () => { ... })` runs commands on the other origin. Better for most tests: log in through the identity provider's API and use `cy.session`.

In the framework: not needed (one origin).

---

6. API testing with cy.request

Q51. How do you test APIs with Cypress?

`cy.request` sends the request from Node and yields status, headers, body and duration. By default a 4xx or 5xx fails the test; pass `failOnStatusCode: false` to assert on error answers.

In the framework: `cypress/support/api.ts` always passes `failOnStatusCode: false` and yields the full response; 31 API tests in `cypress/e2e/api`.

Q52. How do you organise API calls?

A typed client: one function per endpoint, generic response types, the token passed in. Tests read like `api.updateTask(token, id, { done: true })` and never build URLs or headers themselves.

In the framework: the `api` object in `cypress/support/api.ts`.

Q53. What is contract testing at the response level, and how do you do it?

Checking the **shape** of a response (fields, types, formats, no extra fields), not only some values. With zod:

```
export const TaskSchema = z.strictObject({
  id: z.string().min(1),
  priority: z.enum(['low', 'medium', 'high']),
  dueDate: z.iso.date().nullable(),
  /* ... */
});
expectSchema(TaskSchema, response.body);
```

`strictObject` fails on unknown fields, so a leaked field (password, internal id) is caught.

In the framework: `cypress/support/schemas.ts`; checked by leaking `ownerId` on purpose: `Unrecognized key: "ownerId"`.

Q54. Which negative API tests matter most?

Missing and wrong-type fields, boundary lengths, invalid formats (dates), unknown fields, invalid JSON, missing or invalid token, and **access to another user's data**. For each: the status, the message, the field, and that **nothing was saved**.

In the framework: `cypress/e2e/api/validation.cy.ts` (data-driven table) and `auth.cy.ts`.

Q55. Why 404 and not 403 for another user's resource?

403 tells an attacker the id exists. 404 reveals nothing. Same idea: "wrong username or password" for both an unknown user and a wrong password.

In the framework: both are asserted in `cypress/e2e/api/tasks.cy.ts` and `auth.cy.ts`.

Q56. How do you write data-driven tests in Cypress?

Generate tests from an array with `forEach` around `it`: each case is its own test with its own name in the report. Keep the data next to the test, or in a fixture when it is large.

In the framework: login bodies (`auth.cy.ts`), validation cases (`validation.cy.ts`), title boundaries (`TaskForm.cy.tsx`).

Q57. When would you not use Cypress for API tests?

For large API suites with no UI, load tests or contract tests between services. A dedicated tool (REST Assured, pytest + requests, k6, Pact) is lighter. Cypress API tests are ideal next to UI tests: same language, same reports, and the same client for setup data.

In the framework: API tests share the client with UI setup (`cy.createTask`). My portfolio also has REST Assured, pytest and k6 repositories.

---

7. Component testing

Q58. What is Cypress component testing?

Mounting one component in a real browser, with the app's bundler (Vite/webpack) and styles, and testing it with the normal Cypress API. Faster than E2E, more realistic than jsdom unit tests.

In the framework: 17 tests in `cypress/component`; `devServer: { framework: 'react', bundler: 'vite' }`.

Q59. How do you test callbacks (props) of a component?

Pass `cy.stub()`s and assert what the component asked for:

```
const onCreate = cy.stub().as('create').resolves();
cy.mount(<TaskForm onCreate={onCreate} />);
// ...
cy.get('@create').should('have.been.calledOnceWith', { title: 'Plan', priority: 'low', dueDate:
null });
```

In the framework: `TaskForm.cy.tsx`, `TaskItem.cy.tsx`.

Q60. How do you test a busy or pending state in a component test?

Give the component a promise the test controls and resolve it later:

```
let finish = () => undefined;
const onCreate = cy.stub().returns(new Promise<void>((resolve) => (finish = resolve)));
// click, assert disabled ...
cy.then(() => finish());
```

In the framework: "disables the button while saving" (`TaskForm.cy.tsx`).

Q61. How do you test a controlled component that needs a parent?

Write a small harness component in the spec that holds the state like the real parent.

In the framework: `Harness` in `cypress/component/FilterBar.cy.tsx` .

Q62. Component test or E2E test?

Component tests: rules and states of one component (boundaries, rendering variants, callbacks). E2E: that the pieces are wired together and the user flow works. Avoid testing the same detail twice: details in the component test, one wiring check in E2E.

In the framework: title boundaries in `TaskForm.cy.tsx` ; one E2E check that no request is sent for a short title (`tasks.cy.ts`).

Q63. What does a component spec need to run?

`component` config with the dev server, a `component-index.html` mount page, a support file that registers `cy.mount` (from `cypress/react`) and imports global styles.

In the framework: `cypress/support/component-index.html` , `cypress/support/component.ts` .

---

8. Configuration, TypeScript and CI

Q64. What goes into `cypress.config.ts` ?

Global options (reporter, viewport, timeouts, retries, video), `env` and `expose` values, and one section per testing type: `e2e` (`baseUrl` , `specPattern` , `setupNodeEvents`) and `component` (`devServer` , `specPattern`). `setupNodeEvents` can change the resolved config and must return it.

In the framework: `cypress.config.ts` ; `setupNodeEvents` excludes the demo specs and sets a separate report folder for component runs.

Q65. How do you pass secrets and settings in Cypress 16?

Cypress 16 separates them:

- `env` : **secret** values. Read with `cy.env(['password'])` , which yields them to the test; they are not bundled into the browser code. Set them with `CYPRESS_<NAME>` environment variables in CI.
- `expose` : **public** values, readable everywhere with `Cypress.expose('name')` , set with `--expose name=value` on the command line.

In the framework: `env.password` (overridable with `CYPRESS_PASSWORD`), `expose.demo` (`npm run cy:demo`).

Q66. How do you run against several environments?

One config with a `baseUrl` that comes from the environment (`CYPRESS_BASE_URL` or `--config baseUrl=...`), secrets per environment from the CI's secret store, and data that each environment can

create through its API (no hard-coded ids from one environment).

In the framework: `baseUrl` is localhost; tests create their own data and use seeded names, so a different `baseUrl` is the only change.

Q67. How do you set up TypeScript for Cypress?

A `cypress/tsconfig.json` with `"types": ["cypress", "node"]`, `strict`, and for component tests `"jsx": "react-jsx"`. Cypress bundles the specs itself; `tsc --noEmit` is only for type checking in CI. Custom commands get types through `declare global { namespace Cypress { interface Chainable ... } } }`.

In the framework: `cypress/tsconfig.json`; `npm run typecheck` checks app and tests.

Q68. Which ESLint rules matter for Cypress?

From `eslint-plugin-cypress`: `no-unnecessary-waiting` (no `cy.wait(ms)`), `no-assigning-return-values`, `unsafe-to-chain-command`, `no-force`, `no-pause`. With typescript-eslint's type-aware rules on top.

In the framework: `eslint.config.js`, all as errors; CI's first job runs them.

Q69. How do you run Cypress in GitHub Actions?

`cypress-io/github-action` installs dependencies (with the Cypress binary cache), runs `build`, starts the app with `start`, waits for `wait-on`, runs the specs and writes a job summary. A matrix runs browsers in parallel; reports and screenshots are uploaded as artifacts, also when tests fail.

In the framework: `.github/workflows/ci.yml`: checks, then API, UI (Chrome, Firefox, Edge) and component jobs.

Q70. How do you speed up a Cypress suite?

- Log in and create data through the API (`cy.session`, `cy.request`).
- Test rules in component or API tests, keep E2E for flows.
- Split specs across parallel CI jobs (by folder or browser; Cypress Cloud balances specs automatically).
- Skip the binary install where it is not needed (`CYPRESS_INSTALL_BINARY=0`).
- No fixed waits.

In the framework: all of these: the full E2E run (64 tests) takes about 30 s locally; the checks job does not download Cypress.

Q71. How do you run only part of the suite?

`--spec` with a glob (`cypress/e2e/api/**`), `excludeSpecPattern`, or tags with a grep plugin. Folders are the simplest.

In the framework: `npm run cy:api`, `cy:ui`, `cy:component`. A tag plugin was left out because it pulled in a dependency with an open high-severity advisory; folders give the same split.

Q72. Videos and screenshots?

Screenshots are taken automatically on failure in `cypress run`. Videos are off by default; switch them on with `video: true` (or `CYPRESS_VIDEO=true`) when they are worth the extra time.

In the framework: `video: false`, screenshots embedded in the report; see `docs/images/failure-screenshot.png`.

---

9. Flaky tests, debugging and reporting

Q73. What are the usual causes of flaky Cypress tests?

- Fixed waits instead of waiting on a state or request.
- Chaining after actions on elements that re-render.
- Shared data between tests or with other people (public demo sites).
- Assertions on texts that differ by browser, time zone or date.
- Animations and overlays (actionability helps, `force` hides the problem).
- Missing waits on the request that loads the data before asserting "nothing".

In the framework: each has a rule: lint for waits and chains, data reset per test, frozen clock, browser-specific messages not asserted.

Q74. How do you debug a failing test?

Run it in `cypress open`, read the error, hover the commands to time-travel, click a command for details in DevTools, use `.debug()` or `cy.pause()` locally (never commit `pause`). In CI, open the report and the failure screenshot; for timing problems, switch videos on.

In the framework: `cypress/no-pause` is a lint error; screenshots are in the HTML report.

Q75. Should you use test retries?

As a safety net in CI, yes (one retry); never to hide a known flaky test. Report retries and fix the cause. Locally, no retries, so flakiness is visible.

In the framework: `retries: { runMode: 1, openMode: 0 }`; the local verification runs used `--config retries=0`.

Q76. How do you handle uncaught exceptions from the app?

By default an uncaught exception fails the test, which is usually right: it is a bug. If a third-party script throws something irrelevant, ignore only that error: `Cypress.on('uncaught:exception', (err) => !err.message.includes('ThirdPartyWidget'))`. Never ignore all.

In the framework: not needed: the app has no third-party scripts, so every exception would fail a test.

Q77. Which reporters do you use?

The built-in `spec` reporter for the console, JUnit XML for CI test tabs, and an HTML report for people (mochawesome with embedded screenshots). Cypress Cloud adds history and flakiness tracking.

In the framework: `cypress-mochawesome-reporter`, one report per run in `cypress/reports/e2e` and `cypress/reports/component`, uploaded by CI.

Q78. How do you prove a test can fail?

Break the product on purpose (or the expectation) and watch the test fail with a clear message, then restore it. A test that never failed may test nothing.

In the framework: the contract test was checked by leaking `ownerId` from the server; the report demo (`npm run cy:demo`) fails on purpose to show the failure output.

---

10. Framework design and scenarios

Q79. Page objects or app actions in Cypress?

Page objects keep selectors and screen steps in one place; "app actions" call the app or API directly to reach a state. Use both: app actions (API, `cy.session`) for setup, page objects for the screen under test. Keep page objects thin: methods return Cypress chains, assertions stay in the specs.

In the framework: `cypress/pages/*` + API setup through `cy.createTask` and `cy.loginAs`.

Q80. How is this framework layered?

Specs (behaviour) → page objects (screens) → commands (cross-cutting setup) → API client (`cy.request`) → contracts (zod) → test data (seeded names, builders). Component specs mount app components directly.

In the framework: `docs/framework-design.md`, section 3.

Q81. How do you keep test data under control?

Reset to known seed data before each test, create test-specific data through the API, give it unique names, refer to seeded data by name from one place, and never depend on order between tests.

In the framework: `POST /api/test/reset`, `cypress/support/data.ts` (`users`, `seeded`, `buildTask()`).

Q82. A test passes locally and fails in CI. What do you check?

Timing (CI machines are slower: wait on states, not times), screen size (headless default viewport), time zone and date, data left by other jobs, environment variables and secrets, browser version. Read the CI report and screenshot first, then reproduce with the same browser headless.

In the framework: fixed viewport, frozen clock, own server per CI job, secrets from `CYPRESS_*`.

Q83. How would you test the search feature of an app?

API: filtering rules (case, partial match, combined filters, invalid parameters). UI: the request carries the right parameters, the debounce (`cy.clock`), results and the "no results" state, combination with other filters.

In the framework: `api/tasks.cy.ts` (filters), `ui/filters.cy.ts` (query parameters, debounce, combinations, empty state).

Q84. How would you test an error state that the real backend rarely produces?

Stub it with `cy.intercept` (500, network error, slow answer, 401), assert the message and the recovery path (Retry, back to login), and keep one test against the real backend for the happy path.

In the framework: `cypress/e2e/ui/network.cy.ts`.

Q85. Why does this project test its own app instead of a public demo site?

Reliability (nothing outside the repo can break CI), control (reset hook, seeded data, exact rules) and range (component tests need the source). The trade-off, maintaining the app, is small: the API uses only Node built-ins and has its own tests.

In the framework: `app/` and `docs/framework-design.md`, section 2.

Q86. Tell me about this framework in two minutes.

"It is a Cypress 16 + TypeScript framework for a React and Node task app that lives in the same repo. There are three layers of tests: 31 API tests with a typed `cy.request` client and zod contracts that fail on any

unexpected field; 33 UI tests with page objects that use the real backend for flows and `cy.intercept` and `cy.clock` for error states, slow responses, debounce and date logic; and 17 component tests that mount the React components with stubbed callbacks. Every test resets the data and logs in through a cached `cy.session`. Lint bans fixed waits and unsafe chains. GitHub Actions runs lint and types first, then API, UI in Chrome, Firefox and Edge, and component tests in parallel, with an HTML report per job."

In the framework: README, "What it shows".

---

11. Quick reference

Need	Cypress
element by test id	`cy.get('[data-cy="add-task"]')` (here: `cy.getBySel('add-task')`)
element by text	`cy.contains('[data-cy="task-title"]', /^Exact title$/)`
scope	`.within(() => { ... })`
wait for request	`cy.intercept(...).as('x'); cy.wait('@x')`
no request sent	`cy.get('@x.all').should('have.length', 0)`
stub response	`cy.intercept('GET', url, { statusCode: 500, body })`
first call only	`cy.intercept({ method, url, times: 1 }, response)`
delay response	`req.on('response', (res) => res.setDelay(1000))`
fixed date	`cy.clock(timestamp, ['Date'])`
control timers	`cy.clock(Date.now(), ['setTimeout', 'clearTimeout']); cy.tick(300)`
cached login	`cy.session(id, setup, { validate, cacheAcrossSpecs: true })`
storage before load	`cy.visit(url, { onBeforeLoad(win) { ... } })`
API call	`cy.request({ method, url, body, failOnStatusCode: false })`
secret value	`cy.env(['password']).then(({ password }) => ...)`
public value	`Cypress.expose('demo')`, `--expose demo=true`
confirm dialog	`cy.on('window:confirm', () => false)`
callback spy	`cy.stub().as('x')`, `should('have.been.calledOnceWith', ...)`
mount component	`cy.mount(<TaskForm onCreate={stub} />)`
specs by folder	`cypress run --spec "cypress/e2e/api/**"`
browser	`cypress run --browser firefox`